

Informe Preliminar — Desafío 1: Implementación *Sweet Crush* mediante manipulación de bits en C++

Jharlin Castro Moreno – jharlin.castro@udea.edu.co

Informática II — Semestre 2026-2
Universidad de Antioquia
Septiembre de 2026

Repositorio: <https://github.com/jhacasky1524/Desafio1-SweetCrush-InformaticaII.git>

Índice

1. Contextualización	2
1.1. Objetivo general del proyecto	2
1.2. Alcance de esta entrega	2
2. Análisis	3
2.1. Análisis del problema	3
2.2. Codificación de las fichas (3 bits por posición)	3
2.3. Representación compacta del tablero en memoria	3
2.3.1. Cálculo del índice y ubicación de una ficha	3
2.3.2. Fichas repartidas entre dos bytes	3
2.4. Cálculo de memoria requerida	4
2.5. Detección de combinaciones	5
2.6. Cascadas y reorganización	5
2.7. Modificación dinámica del tablero (filas/columnas)	5
2.8. Política de liberación de memoria (regla del 65 %)	5
2.9. Restricciones de diseño del lenguaje	6
3. Diseño	8
3.1. Arquitectura general de la solución	8
3.2. Organización modular (archivos .h / .cpp)	8
3.3. Esquema de tareas / algoritmos definidos	8
3.4. Selección de tipos de datos y estructuras	8
3.5. Criterio de puntuación definido	9
3.6. Plan de trabajo hacia la entrega final	9

1. Contextualización

”normal”. Menciona también, brevemente, quién conforma el equipo y cómo se organizó el trabajo.

El desafío consiste en desarrollar un juego tipo *match-3* en el que las fichas deben almacenarse mediante una representación compacta en bits. A diferencia de un *match-3* convencional, donde cada ficha puede ocupar una posición de memoria completa, aquí varias fichas comparten los bytes disponibles, haciendo que la gestión y el redimensionamiento eficiente de la memoria sean parte fundamental del problema.

1.1. Objetivo general del proyecto

La restricción de bits compactos hace que el reto principal no sea únicamente implementar la lógica del juego, sino administrar eficientemente la memoria para almacenar y redimensionar el tablero sin desperdiciar espacio. Por eso el objetivo del proyecto se centra en tres frentes que corresponden directamente a los criterios de evaluación del desafío: el uso correcto de operadores a nivel de bits para localizar y manipular cada ficha dentro de la trama compacta, la gestión eficiente de memoria dinámica al crear y redimensionar el tablero, y la lógica de juego (detección de combinaciones, cascadas y puntuación) construida sobre esa representación compacta, sin recurrir a estructuras auxiliares que desempaqueten permanentemente el tablero.

1.2. Alcance de esta entrega

Esta entrega corresponde al informe preliminar, que cubre la Contextualización, el Análisis y el Diseño de la solución propuesta. La evidencia del proceso de implementación (código fuente completo, repositorio actualizado y video de sustentación) se entregará el 18 de septiembre de 2026, según lo establecido en el cronograma del desafío.

2. Análisis

2.1. Análisis del problema

Las restricciones que más complican el desarrollo son el almacenamiento compacto de las fichas, ya que una ficha puede quedar dividida entre dos bytes, y el redimensionamiento dinámico de la memoria, porque debe realizarse sin perder información y aplicando la regla del 65 %. Además, la prohibición de utilizar `struct`, `class`, `template` y STL obliga a manejar por separado los datos del tablero y aumenta la complejidad de las funciones.

2.2. Codificación de las fichas (3 bits por posición)

Código (3 bits)	Significado	Símbolo en pantalla
000	Ficha A	A
001	Ficha B	B
010	Ficha C	C
011	Ficha D	D
100	Ficha E	E
101	Ficha F	F
110	Estado especial	*
111	Posición vacía	.

y .): ¿mejora la legibilidad del tablero en consola/Qt? ¿Facilita distinguir a simple vista una ficha normal de un estado especial o vacío?

Usar letras como A-F y símbolos como * y . permite identificar rápidamente las fichas y los espacios vacíos sin tener que interpretar largas secuencias de código binario.

2.3. Representación compacta del tablero en memoria

El tablero se representa como una única secuencia continua de bits, donde cada ficha ocupa solamente la cantidad de bits asignada (3 bits), sin reservar un byte completo para cada posición. Por esto, una ficha puede comenzar en un byte y terminar en el siguiente, como se ilustra en la Figura 3 del enunciado.

2.3.1. Cálculo del índice y ubicación de una ficha

$$\text{índice} = \text{fila} \times C + \text{columna} \quad (1)$$

$$\text{bit_inicial} = \text{índice} \times 3 \quad (2)$$

$$\text{byte_inicial} = \left\lfloor \frac{\text{bit_inicial}}{8} \right\rfloor \quad (3)$$

$$\text{offset} = \text{bit_inicial} \bmod 8 \quad (4)$$

2.3.2. Fichas repartidas entre dos bytes

Un byte tiene 8 bits, numerados del 0 al 7. Si una ficha ocupa 3 bits consecutivos, debemos asegurarnos de que esos 3 bits no se salgan del byte. La ficha comienza en `offset` y ocupa:

$$\text{offset}, \text{offset} + 1, \text{offset} + 2$$

Por lo tanto, para que los tres bits estén dentro del mismo byte, el último bit debe ser como máximo el bit 7:

$$\text{offset} + 2 \leq 7$$

De ahí obtenemos:

$$\text{offset} \leq 5$$

Esto significa que:

- Si `offset` está entre 0 y 5, los 3 bits caben completamente en el mismo byte.
- Si `offset` = 6, la ficha ocupa los bits 6, 7 y 8. Los bits 6 y 7 están en el byte actual, pero el bit 8 pertenece al siguiente byte.
- Si `offset` = 7, la ficha ocupa los bits 7, 8 y 9, por lo que solo el primer bit queda en el byte actual y los otros dos pasan al siguiente.

Por eso, en el código se utiliza la condición `offset > 5`. Esta condición detecta precisamente los casos `offset` = 6 y `offset` = 7, que son los únicos en los que la ficha no cabe completamente en un solo byte y es necesario utilizar también el byte siguiente.

La idea es no hacer un `if` para decidir *cómo* extraer la ficha. En lugar de eso, se lee el byte actual y el siguiente juntos, formando una especie de "ventana" de 16 bits:

```
1 ventana = byte_actual | (byte_siguiente << 8);
```

Así, aunque la ficha esté completamente dentro del primer byte o esté dividida entre los dos bytes, se puede tratar como si estuviera en una secuencia continua de bits. Después, se realizan dos operaciones:

```
1 ficha = (ventana >> offset) & 0b111;
```

Primero, `offset` mueve los bits de la ficha hasta el comienzo, es decir, hasta la posición 0. Luego, `& 0b111` conserva únicamente los 3 bits de la ficha y elimina todos los demás.

Por ejemplo, si `offset` = 6, la ficha ocupa los bits 6 y 7 del primer byte y el bit 0 del siguiente. Como ambos bytes se juntan en la ventana, esos tres bits quedan seguidos y se pueden extraer exactamente con la misma fórmula. Por eso no se necesita un `if` para extraer la ficha: la misma operación funciona para cualquier `offset` entre 0 y 7.

La condición `if (offset > 5)` se utiliza después, cuando se va a *escribir* la ficha en memoria. Ahí sí es necesario saber si los 3 bits caben en el primer byte o si también debe modificarse el siguiente.

2.4. Cálculo de memoria requerida

$$\text{bytes_necesarios} = \left\lceil \frac{3 \times F \times C}{8} \right\rceil \quad (5)$$

Para un tablero de 7 filas $\times$ 6 columnas se tienen 42 posiciones, es decir $3 \times 42 = 126$ bits necesarios.

Con 15 bytes solo se dispone de $15 \times 8 = 120$ bits, pero se necesitan 126 bits para las 42 fichas; por eso no alcanza y debe usarse 16 bytes, que proporcionan $16 \times 8 = 128$ bits, dejando 2 bits sobrantes (bits "inválidos" que, según el enunciado, deben agruparse al final de la trama).

2.5. Detección de combinaciones

El segundo mapa de marcado permite marcar primero todas las fichas que deben eliminarse sin borrarlas inmediatamente. Así, una ficha que pertenece a una combinación horizontal y también vertical queda simplemente marcada una vez; al final se eliminan todas juntas sin perder información durante el recorrido.

Se recorre cada fila y columna manteniendo un contador de fichas consecutivas iguales. Cuando la ficha cambia o se llega al final del recorrido, si el contador es mayor o igual a 3, se marcan en el mapa de combinaciones todas las posiciones correspondientes a ese tramo para eliminarlas posteriormente.

2.6. Cascadas y reorganización

Una cascada consiste en repetir el proceso de eliminar, hacer caer las fichas y rellenar los espacios con fichas nuevas, y después volver a revisar si se formó otra combinación. El ciclo continúa mientras aparezcan nuevas combinaciones automáticamente y se detiene cuando el tablero ya no tiene ninguna combinación nueva.

Para cada columna se recorre el tablero de abajo hacia arriba, buscando las fichas que no estén marcadas como vacías. Cada ficha encontrada se coloca en la posición vacía más baja disponible. Al terminar el recorrido, los espacios que queden libres se mantienen en la parte superior de la columna y posteriormente pueden rellenarse con nuevas fichas. En términos conceptuales:

1	Antes:		Despues de caer:
2			
3	A . C		A . .
4	. B .	->	. . .
5	D . E		D B E

La idea es compactar todas las fichas hacia abajo sin cambiar su orden relativo.

2.7. Modificación dinámica del tablero (filas/columnas)

El índice se calcula como $\text{fila} \times \text{número_de_columnas} + \text{columna}$, así que al cambiar el número de columnas cambia el salto que se hace de una fila a la siguiente. Por ejemplo, en un tablero de 2×3 , los índices son 0,1,2,3,4,5; si pasa a 2×4 , quedan 0,1,2,3,4,5,6,7, por lo que las fichas de la segunda fila ya no están en las mismas posiciones. Por eso hay que recalcular y reconstruir la posición de las fichas en la nueva trama.

Cuando se agrega una fila o columna, la nueva sección se llena con fichas aleatorias, ya que debe incorporarse como parte funcional del tablero. Cuando se elimina una fila o columna intermedia, se reconstruye la representación del tablero copiando únicamente las posiciones que permanecen y omitiendo la fila o columna eliminada. Después se actualizan las dimensiones y se calcula nuevamente la memoria necesaria para determinar si debe aplicarse la regla del 65 %.

2.8. Política de liberación de memoria (regla del 65 %)

La idea es mantener dos valores:

1. **Memoria reservada:** cuánto espacio se tiene actualmente asignado para el tablero.
2. **Memoria necesaria:** cuánto espacio se necesita realmente con las filas y columnas que quedan después de una eliminación.

Después de eliminar una fila o columna, se calcula nuevamente la memoria necesaria y se compara:

$$\text{memoria_necesaria} < 0,65 \times \text{memoria_reservada}$$

Ejemplo. Supongamos que se tiene una memoria reservada de 1000 unidades y, tras una eliminación, la memoria necesaria baja a 600 unidades. Como $0,65 \times 1000 = 650$ y $600 < 650$, sí se debe reducir la memoria reservada. En cambio, si la memoria necesaria fuera 700, como $700 > 650$, no se redimensiona todavía y se sigue utilizando el espacio de 1000 unidades ya reservado.

```
1 memoriaNecesaria = calcular_bytes(filasActuales , columnasActuales);
2
3 si (memoriaNecesaria < 0.65 * memoriaReservada)
4     reducir_memoria_reservada();
```

La ventaja de este mecanismo es que no se hace un `new` y un `delete` después de cada eliminación. Se espera hasta que realmente haya una cantidad significativa de memoria desperdiciada antes de redimensionar. En otras palabras: reservar memoria es como alquilar un espacio; aunque se elimine algo, no se cambia de apartamento inmediatamente, solo se busca uno más pequeño cuando se está usando menos del 65 % del espacio actual.

La memoria se ajusta exactamente al mínimo necesario cuando se activa el 65 %. Por ejemplo, si `memoriaReservada = 1000` bytes y `memoriaNecesaria = 600` bytes, como $600 < 650$ se activa la regla y se redimensiona a exactamente 600 bytes, porque ese es el espacio que realmente necesita el tablero en ese momento.

No se deja margen adicional porque el enunciado indica que, al caer por debajo del 65 % de lo reservado, se debe reducir la memoria física reservada. La opción más directa y fácil de justificar es `nuevaMemoria = memoriaNecesaria`, con lo cual se mantiene siempre `memoriaReservada >= memoriaNecesaria` y se evita conservar memoria que ya no se está utilizando.

```
1 memoriaNecesaria = calcularMemoria(filas , columnas);
2
3 si (memoriaNecesaria < 0.65 * memoriaReservada) {
4     memoriaReservada = memoriaNecesaria;
5     redimensionarMemoria();
6 }
```

Es importante notar que no se redimensiona simplemente porque se eliminó una fila o columna: primero se verifica la condición del 65 %, y solo entonces se realiza la reasignación de memoria.

2.9. Restricciones de diseño del lenguaje

, `class`, `template`, STL, ni `string`, y qué implicaciones tiene esto sobre el paso de parámetros entre funciones (tablero, dimensiones, estado del juego).

Se usarán principalmente referencias (`&`) para los datos que una función necesita modificar. El tablero se maneja con tres variables separadas, respetando la representación compacta de bits exigida por el enunciado (un único bloque dinámico de `unsigned char`, no una matriz de enteros ni una ficha por byte):

```
1 unsigned char *tablero;
2 int filas;
3 int columnas;
```

Una función como `agregarFila` recibiría el puntero y las dimensiones así:

```
1 void agregarFila(unsigned char *&tablero, int &filas, int columnas);
```

- `unsigned char *&tablero`: permite que la función cambie la dirección del bloque de memoria cuando tenga que reservar uno nuevo (por ejemplo, al redimensionar el tablero).
- `int &filas`: permite modificar el número de filas directamente.
- `int columnas`: no se modifica dentro de esta función, así que se pasa por valor.

Si el puntero al tablero se pasara sin referencia (`unsigned char *tablero`), la función podría modificar los bytes apuntados, pero no podría actualizar correctamente el puntero original cuando se haga una nueva reserva de memoria (el llamador seguiría apuntando al bloque viejo, ya liberado).

Una alternativa sería usar doble puntero (`unsigned char **tablero, int *filas`), pero resultaría más difícil de leer y usar, al tener que desreferenciar constantemente (`*tablero, *filas`). Por eso, dadas las restricciones del ejercicio, la elección es: `tablero` → `unsigned char *&`, `filas` → `int &`, `columnas` → `int` (por valor cuando no se modifica). El mismo criterio se aplicará a las demás funciones: referencia para aquello que la función pueda modificar y paso por valor para lo que solo necesita consultar.

3. Diseño

3.1. Arquitectura general de la solución

La solución se organiza en módulos independientes que separan la gestión de memoria y bits, la lógica del juego y la interacción con el usuario, comunicándose mediante un puntero principal al tablero y sus dimensiones.

3.2. Organización modular (archivos .h / .cpp)

Módulo	Responsabilidad
tablero.h / .cpp	Creación, eliminación y modificación del tablero
memoria.h / .cpp	Cálculo de bits/bytes, reservas y redimensionamiento
juego.h / .cpp	Reglas, movimientos, eliminaciones, cascadas y puntuación
estructura.h / .cpp	Manejo de la estructura compacta del tablero
main.cpp	Menú y flujo principal

Con esta división, cada módulo tiene una responsabilidad clara y delimitada, lo que facilita el mantenimiento del código y respeta el requisito de programación modular/multi-archivos exigido por el enunciado.

3.3. Esquema de tareas / algoritmos definidos

1. Crear y reservar la memoria del tablero.
2. Inicializar el tablero con fichas aleatorias.
3. Mostrar el tablero.
4. Solicitar al jugador la posición de una ficha.
5. Validar el movimiento o selección.
6. Eliminar la ficha o grupo correspondiente.
7. Detectar nuevas combinaciones.
8. Aplicar la caída de fichas y rellenar los espacios.
9. Repetir las eliminaciones mientras existan nuevas combinaciones (cascadas).
10. Actualizar la puntuación.
11. Verificar si la partida continúa.
12. Mostrar el resultado final.

3.4. Selección de tipos de datos y estructuras

- **Tablero:** `unsigned char*`, porque permite manejar directamente la memoria como una secuencia de bytes.
- **Índices y contadores:** `int`, por simplicidad y porque permite trabajar cómodamente con filas, columnas, posiciones y cantidades.

- **Mapa de combinaciones:** `unsigned char*`, utilizando cada posición como una marca para indicar si una ficha debe eliminarse.
- **Generador aleatorio:** `unsigned int` como semilla/estado si se implementa manualmente, junto con operaciones enteras para obtener los valores de las fichas. También podría utilizarse `rand()` de C++ si el enunciado lo permite.

3.5. Criterio de puntuación definido

Se utilizará un sistema de multiplicador por cascada:

$$\text{puntos} = \text{fichas_eliminadas} \times \text{multiplicador}$$

donde el multiplicador aumenta con cada cascada sucesiva producida por la misma eliminación (cascada 1 $\rightarrow \times 1$, cascada 2 $\rightarrow \times 2$, cascada 3 $\rightarrow \times 3$, cascada 4 $\rightarrow \times 4$, y así sucesivamente). Este esquema es sencillo de implementar y además recompensa las cadenas de eliminaciones, incentivando al jugador a generar cascadas.

3.6. Plan de trabajo hacia la entrega final

El trabajo se dividirá por módulos entre los integrantes: uno se encargará principalmente de la gestión de memoria y la representación compacta del tablero, mientras el otro desarrollará la lógica del juego, las combinaciones y la puntuación. Después se integrarán y probarán los módulos conjuntamente. Se harán commits frecuentes, idealmente cada vez que se complete una funcionalidad o corrección importante, evitando acumular demasiados cambios en un solo commit.

Referencias

Para esta entrega se deja como referencia principal el enunciado del desafío y los apuntes/material proporcionados en clase, ya que el diseño debe ajustarse específicamente a las restricciones planteadas. Las consultas externas de C++ solo serían complementarias para resolver dudas puntuales de sintaxis o manejo de memoria.